

# 20 System Design Interview Problems

For Freshers, Internship & Entry-Level Candidates

Scalability | Databases | Caching | Load Balancing | Microservices | Real-World Systems

## SECTION 1: Foundation Problems

### Problem 1: Design a URL Shortener (like bit.ly)

*Most commonly asked system design question. Tests hashing, databases, and scalability.*

#### Functional Requirements

Shorten a long URL. Redirect short URL to original. Optional: analytics (click count), expiry.

#### Core Components

API Layer: POST /shorten, GET /{shortCode}

Hashing: Generate unique 6-7 char code using Base62 encoding (a-z, A-Z, 0-9).

Database: Store mapping shortCode to originalURL. Use Cassandra for scale.

Cache: Redis to cache popular short URLs for fast redirects.

Load Balancer: Distribute traffic across multiple servers.

#### Scalability Points

Use a distributed ID generator (like Snowflake) to avoid collisions.

CDN for static assets. Read-heavy system, so cache aggressively.

Database sharding by shortCode hash for horizontal scaling.

#### Key Concept

CAP Theorem: For this system, prefer Availability + Partition Tolerance (AP) over strong consistency.

### Problem 2: Design a Rate Limiter

*Tests algorithms, distributed systems, and API design.*

#### What is it?

Controls how many requests a client can make in a given time window to prevent abuse and ensure fair usage.

#### Algorithms

Token Bucket: Tokens added at fixed rate; request consumes a token. Allows bursts.

Leaky Bucket: Requests processed at fixed rate; excess queued or dropped.

Fixed Window Counter: Count requests per fixed time window (e.g., 100 req/min).

Sliding Window Log: Most accurate; stores timestamps of each request.

#### Implementation

Use Redis with INCR + EXPIRE for distributed rate limiting.

Store key as user\_id:timestamp and count.

Return HTTP 429 Too Many Requests when limit exceeded.

Add headers: X-RateLimit-Limit, X-RateLimit-Remaining, Retry-After.

#### Key Concept

For distributed systems, centralized Redis cache ensures rate limits work across multiple server instances.

### Problem 3: Design a Parking Lot System

*Classic OOP + system design. Tests object modeling and database design.*

## Entities

ParkingLot, ParkingFloor, ParkingSpot (Compact/Large/Handicapped), Vehicle (Car/Truck/Bike), Ticket, Payment.

## Core Flow

Vehicle enters: Find available spot, issue ticket with entry time.

Vehicle exits: Calculate fee (hourly rate), process payment, free spot.

## Database Design

parking\_spots (id, floor, type, is\_occupied)

tickets (id, spot\_id, vehicle\_number, entry\_time, exit\_time, amount)

payments (id, ticket\_id, amount, method, status)

## Key Concept

Concurrency handling: use optimistic locking or database transactions to avoid race conditions when assigning spots.

## Problem 4: Design a Key-Value Store (like Redis)

*Tests understanding of storage engines, consistency, and distributed systems.*

### Operations

GET(key), PUT(key, value), DELETE(key), optional TTL (expiry).

### Storage

In-memory hash map for speed. Persist to disk using WAL (Write-Ahead Log) or snapshots.

Use LRU (Least Recently Used) eviction when memory is full.

### Distribution

Partition data using Consistent Hashing across nodes.

Replication: each key stored on N nodes for fault tolerance.

Use Gossip Protocol for node health monitoring.

### Key Concept

Consistent Hashing minimizes data reshuffling when nodes are added/removed -- critical for scalable distributed caches.

## Problem 5: Design a Notification System

*Practical problem asked at product companies. Tests async processing and fan-out.*

### Types of Notifications

Push (mobile), Email, SMS, In-app notifications.

### Core Components

API Service: Accepts notification requests.

Message Queue (Kafka/RabbitMQ): Decouples producers from consumers, handles spikes.

Worker Services: Separate workers for push, email, SMS pull from queue.

Template Service: Renders personalized message content.

User Preference Service: Respects opt-out settings per channel.

### Scalability

Fan-out: For broadcasting to millions, push to queue in batches.

Retry mechanism with exponential backoff for failed deliveries.

Priority queues: critical alerts vs. marketing notifications.

### Key Concept

Message queues provide durability and decoupling -- if email provider goes down, messages wait in queue and retry later.

---

## SECTION 2: Real-World Product Design

### Problem 6: Design Twitter / X (Simplified)

*Popular FAANG question. Tests feed generation and scale.*

#### Core Features

Post tweet (280 chars), Follow users, Home timeline (feed), Search tweets.

#### Key Challenge: Feed Generation

Push model (Fan-out on write): When user tweets, push to all followers feeds immediately. Fast read, slow write. Good for users with few followers.

Pull model (Fan-out on read): Fetch tweets from followed users at read time. Slow read, fast write. Good for celebrities.

Hybrid: Push for regular users, pull for celebrities (>1M followers).

#### Storage

Tweets: Cassandra (write-heavy, time-series).

User data: MySQL.

Media (images/videos): Object storage (S3) + CDN.

Feed cache: Redis (sorted set by timestamp).

#### Key Concept

Twitter uses a hybrid fan-out approach. Celebrity tweets with 100M+ followers are pulled on demand, not pre-computed.

---

### Problem 7: Design WhatsApp / Messaging System

*Tests real-time communication, message storage, and delivery guarantees.*

#### Core Features

1-to-1 messaging, group chat, delivery status (sent/delivered/read), media sharing.

#### Real-time Communication

WebSockets: Persistent, bidirectional connection for real-time message delivery.

When recipient is offline, store message in DB and deliver when they reconnect.

#### Message Storage

Use Cassandra: partition by (sender\_id, receiver\_id), cluster by timestamp.

Messages have unique IDs (Snowflake ID for ordering).

End-to-end encryption: keys stored on devices, never on server.

#### Delivery Status

Sent: Server received message.

Delivered: Recipient device received message (ACK sent back).

Read: Recipient opened message (read receipt sent).

#### Key Concept

WhatsApp stores messages only until delivered -- after delivery, messages are deleted from servers. This minimizes storage and enhances privacy.

---

### Problem 8: Design YouTube / Video Streaming

*Tests CDN, video processing pipelines, and storage.*

#### Core Features

Upload video, stream video, search, recommendations, comments, likes.

### Upload Pipeline

User uploads raw video, stored in object storage (S3).

Video Processing Service transcodes to multiple resolutions (360p, 720p, 1080p, 4K) using FFmpeg.

Thumbnails auto-generated. Metadata stored in MySQL. Chunks stored back in S3.

### Streaming

Adaptive Bitrate Streaming (ABR): Player switches quality based on network speed (HLS/DASH protocols).

CDN delivers video chunks from nearest edge server to minimize latency.

### Key Concept

Video transcoding is CPU-intensive. Use message queue (Kafka) to trigger async transcoding workers -- never block the upload API.

---

## Problem 9: Design an E-Commerce Platform (like Amazon)

*Tests database design, inventory management, and order processing.*

### Core Services

Product Catalog, User Service, Cart Service, Order Service, Payment Service, Inventory Service, Notification Service.

### Key Challenges

Inventory: Use optimistic locking or DB transactions to prevent overselling.

Flash sale: Redis to pre-load inventory count; decrement atomically with DECR.

Cart: Store in Redis (fast) with periodic sync to DB.

Orders: Use Saga Pattern for distributed transaction management.

### Database Design

Products: Elasticsearch for search + MySQL for catalog.

Orders: MySQL (ACID transactions critical).

Sessions/Cart: Redis.

### Key Concept

Payment processing should be idempotent -- retrying a payment request must not charge the user twice.

Use idempotency keys.

---

## Problem 10: Design a Search Autocomplete System

*Practical feature design. Tests Trie data structure and caching.*

### How it works

As user types in search box, return top-k matching suggestions in real-time (under 100ms latency).

### Data Structure

Trie (Prefix Tree): Each node stores a character; traverse to find all completions for a prefix.

Each Trie node stores top-k suggestions (by frequency) to avoid full traversal.

### At Scale

Pre-compute top suggestions for each prefix and store in Redis/Memcached.

Update suggestions periodically from search logs using Hadoop/Spark batch jobs.

Separate read (suggestion serving) and write (analytics) paths.

### Key Concept

Real-time Trie updates are expensive. Use two-tier approach: fast cache serves suggestions; background job updates Trie from aggregated logs.

---

## SECTION 3: Infrastructure & Backend Design

### Problem 11: Design a Load Balancer

*Fundamental infrastructure concept. Tests networking and distribution algorithms.*

#### What it does

Distributes incoming traffic across multiple servers to ensure no single server is overwhelmed, improving availability and performance.

#### Algorithms

Round Robin: Requests distributed sequentially. Simple, works when servers are equal.

Weighted Round Robin: Servers with more capacity get more requests.

Least Connections: Route to server with fewest active connections.

IP Hash: Same client always goes to same server (session stickiness).

Random: Simple random selection.

#### Types

Layer 4 (Transport): Routes based on IP/TCP. Fast but less flexible.

Layer 7 (Application): Routes based on URL, headers, cookies. More intelligent.

#### Key Concept

Health checks are critical -- load balancer must periodically ping servers and remove unhealthy ones from rotation automatically.

### Problem 12: Design a Caching System

*Every large-scale system uses caching. Tests cache strategies and consistency.*

#### Why Cache?

Reduce database load, decrease latency, handle traffic spikes. Cache frequently read, rarely updated data.

#### Cache Strategies

Cache-Aside (Lazy Loading): App checks cache first; on miss, loads from DB and populates cache.

Write-Through: Write to cache and DB simultaneously. Always consistent but slower writes.

Write-Back (Write-Behind): Write to cache only; async write to DB. Fast writes, risk of data loss.

Read-Through: Cache sits in front of DB; handles misses automatically.

#### Eviction Policies

LRU (Least Recently Used): Remove least recently accessed item.

LFU (Least Frequently Used): Remove least frequently accessed item.

TTL (Time to Live): Items expire after set duration.

#### Key Concept

Cache invalidation is one of the hardest problems in CS. Design your invalidation strategy (TTL vs event-driven) before writing any code.

### Problem 13: Design a Distributed Message Queue (like Kafka)

*Tests event-driven architecture and async communication.*

#### Core Concepts

Producer: Publishes messages to topics.

Consumer: Subscribes to topics and processes messages.

Broker: Stores and serves messages.

Topic: Category of messages, divided into Partitions for parallelism.

Consumer Group: Multiple consumers sharing a topic's partitions.

### Why Kafka?

- Decouples services -- producer does not wait for consumer.
- Handles traffic spikes -- messages buffered in queue.
- Replay capability -- consumers can re-read old messages.
- Durability -- messages stored on disk with replication.

### Key Concept

Kafka guarantees at-least-once delivery by default. For exactly-once semantics (critical for payments), use idempotent producers and transactional consumers.

---

## Problem 14: Design a CDN (Content Delivery Network)

*Tests knowledge of caching at scale and geographic distribution.*

### What is a CDN?

Geographically distributed network of servers (edge nodes) that cache and deliver content from nearest location to user, reducing latency.

### How it works

- User requests resource (image, video, JS file).
- DNS routes request to nearest edge server.
- If edge has content (cache hit): serve directly.
- If not (cache miss): fetch from origin server, cache it, serve to user.

### Use Cases

Static assets (images, CSS, JS), video streaming, software downloads, DDoS protection (absorbs traffic at edge).

### Key Concept

CDNs use Anycast routing -- multiple servers share the same IP address, and network automatically routes to the nearest one.

---

## Problem 15: Design a Database Sharding Strategy

*Critical for any large-scale data storage discussion.*

### What is Sharding?

Horizontal partitioning of a database -- splitting data across multiple database instances (shards) so no single DB handles all the load.

### Sharding Strategies

- Range-based: Shard by value range (e.g., user\_id 1-1M on shard 1). Simple but can cause hotspots.
- Hash-based: Apply hash function to key; result determines shard. Even distribution.
- Directory-based: Lookup table maps keys to shards. Flexible but lookup table is a bottleneck.
- Geographic: Shard by user's region. Good for data residency compliance.

### Challenges

- Cross-shard queries: Joins across shards are expensive.
- Rebalancing: Adding new shards requires data migration.
- Hotspot: One shard getting disproportionate traffic.

### Key Concept

Consistent Hashing solves the rebalancing problem -- when a shard is added, only  $K/n$  keys need to be remapped ( $K$ =keys,  $n$ =shards).

---

## SECTION 4: Advanced Design Problems

## Problem 16: Design a Ride-Sharing App (like Uber)

*Complex real-world design. Tests geospatial indexing and matching algorithms.*

### Core Features

Request ride, match driver, real-time tracking, fare calculation, payment.

### Location Service

Drivers send GPS location every 4 seconds, stored in Redis with Geospatial index (GEOADD).

On ride request, query nearby drivers within radius using GEORADIUS command.

Use WebSockets or long polling for real-time driver location updates on map.

### Matching Algorithm

Find top-N nearby available drivers.

Rank by ETA (estimated time of arrival), rating, acceptance rate.

Send request to top driver; if no response in 10s, move to next.

### Fare Calculation

Base fare + (per km rate x distance) + (per min rate x time) + surge multiplier.

Surge pricing: When demand > supply in a zone, multiply base fare (1.5x, 2x, etc.).

Store completed trip data in MySQL for billing history.

### Key Concept

Uber uses a hexagonal grid (H3) to partition the map into cells. Drivers are indexed by cell ID, making nearby-driver queries extremely fast.

## Problem 17: Design a Hotel Booking System (like Booking.com)

*Tests inventory management, concurrent booking, and search.*

### Core Features

Search hotels by location/date, view availability, book room, manage reservations.

### Concurrency Challenge

Multiple users booking the same last room simultaneously -- must prevent double booking.

### Solutions

Pessimistic Locking: Lock the room record during booking transaction. Safe but slow.

Optimistic Locking: Use version number; if version changed by commit time, retry.

Redis Lock: Use SETNX (Set if Not Exists) to acquire a distributed lock on the room.

### Database Design

hotels (id, name, location, rating, amenities)

rooms (id, hotel\_id, type, price\_per\_night)

room\_availability (hotel\_id, room\_type, date, total\_rooms, booked\_rooms)

bookings (id, user\_id, room\_id, check\_in, check\_out, status, price)

### Key Concept

Index hotels in Elasticsearch with geolocation -- enables fast 'hotels near me' queries with filters for price, rating, amenities.

## Problem 18: Design a Leaderboard System

*Clean, focused problem. Tests Redis sorted sets and real-time ranking.*

### Requirements

Track user scores, update score in real-time, display top-N users, show a specific user's rank.

### Solution with Redis Sorted Set

ZADD leaderboard [score] [userId]: Add or update user score.

ZREVRANK leaderboard [userId]: Get user's rank (0-indexed, descending).

ZREVRANGE leaderboard 0 9 WITHSCORES: Get top 10 users.

All operations are  $O(\log N)$  -- fast even for millions of users.

### Persistence

Redis is in-memory; replicate to MySQL periodically.

For historical leaderboards (weekly, monthly), archive old data to a data warehouse.

### Scaling

Partition leaderboards by game or region to distribute load.

Use Redis Cluster for horizontal scaling when single instance is not enough.

Batch score updates using Lua scripts to reduce round trips.

### Key Concept

Redis Sorted Sets are the perfect data structure for leaderboards -- they maintain order automatically and support rank queries in  $O(\log N)$ .

## Problem 19: Design a File Storage System (like Google Drive / Dropbox)

*Tests file chunking, deduplication, and sync protocols.*

### Core Features

Upload/download files, sync across devices, share files/folders, version history.

### Upload Strategy

Split large files into chunks (e.g., 4MB each).

Each chunk hashed (SHA-256); if hash exists in storage, deduplication (don't re-upload).

Upload chunks in parallel for speed.

Store chunk metadata (file\_id, chunk\_order, chunk\_hash) in DB.

### Storage Architecture

File chunks: Object storage (S3/GCS).

Metadata: MySQL (files, folders, versions, sharing permissions).

Sync state: Redis (which chunks each client has).

Search: Elasticsearch for full-text file name search.

### Sync Protocol

Long polling or WebSockets to notify clients of changes.

Client compares local chunk hashes with server; only sync changed chunks (delta sync).

### Key Concept

Chunking + deduplication can reduce storage by 20-40%. Two users uploading the same file only store one copy on disk.

## Problem 20: Design a Distributed Task Scheduler (like Cron at Scale)

*Great for backend roles. Tests job scheduling, fault tolerance, and distributed coordination.*

### Requirements

Schedule tasks at specific times or intervals. Handle millions of tasks. Guarantee each task runs at least once. Handle failures with retries.

### Components

Scheduler Service: Accepts job definitions (cron expression, task payload).

Job Store: Database of all scheduled jobs with next\_run\_time.

Trigger Engine: Continuously polls DB for jobs due in next N seconds; pushes to queue.

Worker Pool: Pulls jobs from queue, executes them, updates status.

Message Queue (Kafka/RabbitMQ): Buffers jobs between trigger engine and workers.

## Fault Tolerance

If a worker crashes mid-job: heartbeat timeout detected, job requeued.

Idempotent task design ensures safe retries.

Leader election (using ZooKeeper/etcd) ensures only one trigger engine polls at a time.

## Monitoring

Track job execution history, success/failure rates, average execution time.

Alert on jobs that have not run within expected time window.

Dead letter queue for jobs that fail after max retries.

## Key Concept

Polling DB every second for due jobs does not scale. Use a time-sorted index on `next_run_time` and query only jobs in near-future window. Use Redis for sub-second scheduling.

---

## System Design Interview Strategy (5-Step Framework)

**Step 1 - Clarify Requirements (2-3 min):** Ask: Who are the users? What are the core features? What scale -- DAU, QPS, storage? Read-heavy or write-heavy? Any latency/consistency requirements?

**Step 2 - Estimate Scale (2 min):** Calculate: QPS (queries per second), storage needed per year, bandwidth. Example: 10M DAU, each posts 1 tweet/day = ~116 writes/sec. This drives architecture decisions.

**Step 3 - High-Level Design (5-8 min):** Draw the big picture: Client -> API Gateway -> Services -> Database -> Cache. Identify the main components and how they interact.

**Step 4 - Deep Dive (10-15 min):** Go deep on 2-3 critical components. Discuss trade-offs. Show you understand the hard parts like sharding, caching, or consistency.

**Step 5 - Identify Bottlenecks (3-5 min):** Where will the system fail at scale? How do you handle traffic spikes? Single points of failure? Data loss scenarios? How to monitor and alert?

## Key Concepts to Always Know

>> CAP Theorem: You can only have 2 of: Consistency, Availability, Partition Tolerance.

>> Horizontal vs Vertical Scaling: Scale out (more servers) vs scale up (bigger server).

>> SQL vs NoSQL: When to use each; trade-offs between ACID and eventual consistency.

>> Consistent Hashing: For distributing data across nodes with minimal reshuffling.

>> Message Queues: For async processing, decoupling, and handling traffic spikes.

>> Caching Strategies: Cache-aside, write-through, write-back. Know when to use each.

>> Database Indexing: B-tree vs hash indexes. Composite indexes. Query optimization.

>> Load Balancing Algorithms: Round robin, least connections, consistent hashing.

>> Idempotency: Critical for payment systems -- same request should not have duplicate effects.

>> Eventual Consistency: In distributed systems, data will be consistent eventually, not immediately.

Best of luck with your System Design interviews! ■